



Machine learning with LookupProblemSolver

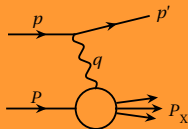
JAROSLAV ADAM

202605.06

- Associative machine learning tool implemented in standard C++ with use of ROOT classes
- n independent quantities are associated with m -dimensional solution
- Simple implementation (~ 500 lines for main part), good performance
- Training is done by providing known solutions for sets of independent quantities
- Solutions can then be obtained for specified values of quantities
- Inside, the quantities are binned for unique index for any set of quantities
- The index holds link to the most probable solution to corresponding quantities
- During training, inputs are stored for each link, then most likely solution is calculated
- For obtaining the solution, it is retrieved via the index
- Several layers with different binning can be created, optional non-uniform binning is possible for efficient sampling of the quantities

- Electron Ion Collider (EIC) will collide electrons with protons (or ions) up to 18x275 GeV

Electron-proton scattering

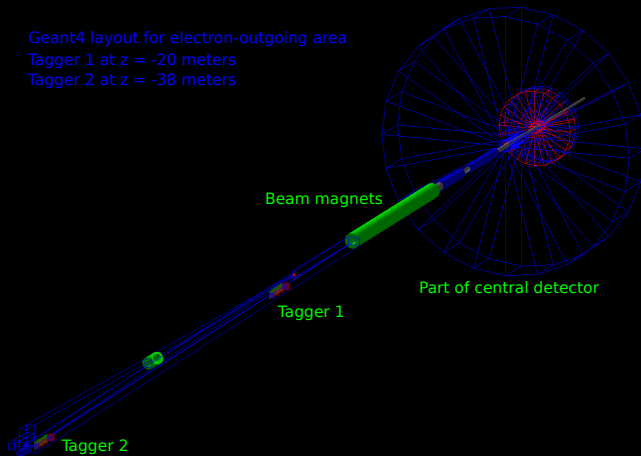


Geant4 layout for electron-outgoing area

Tagger 1 at $z = -20$ meters

Tagger 2 at $z = -38$ meters

- Photoproduction electrons at small virtuality (Q^2) pass through beam magnets
- Complicated quadrupole and dipole magnetic fields before detection in Taggers



- Original electron p' , in terms of energy, θ and ϕ angles can be obtained from tracks via LPS

- We'll test the approach with analytic function (known answer) and then apply it to physics problem of tracks in low- Q^2 tagger
- All the codes are located in `/data2/vyuka/szd/LookupProblemSolver/`
- The tasks will run in Python with use of ROOT classes (PyROOT)
- Outline:
 - 01 | Building the LPS library with cmake
 - 02 | Test with simple analytic function (2-dimensional Gaussian)
 - 03 | Application to tagger tracks
 - 04 | Overview of LPS internals

- There is several codes for LPS which need to be compiled with cmake
- Shell for cmake and running the codes is launched as: `/home/adamjaro/eic/eic-shell`
- The steps are as follows

```
cd LPS_library
mkdir build
cd build
cmake ../
make
```

- LPS implementation is in `LookupProblemSolver.cxx/h`
- Other classes, FoamCellFinder, MedCellFinder and SumCellFinder implement non-uniform binning schemes

Testing the LookupProblemSolver with 2-dimensional Gaussian

- The aim is to train the LPS with values from 2-D Gaussian and confirm that solving gets the same function
- All codes are in Gauss2D directory
- Files `training.py` and `solve.py` provide basic structure for both parts

- Executable file `training.py` :

```
#!/usr/bin/env python3

from ROOT import TF2

def main():

    #two-dimensional Gaussian as a training function
    gauss2 = TF2("gauss2", "exp(-(x^2)-(y^2))", -3, 3, -3, 3)

if __name__ == "__main__":

    #load shared library for the LPS
    gSystem.Load("../LPS_library/build/libLPS.so")

    #boost
    gInterpreter.Declare('#include <boost/program_options.hpp>')

    #header for LPS
    gInterpreter.AddIncludePath("../../../../../reconstruction/include/");
    gInterpreter.Declare('#include "LookupProblemSolver.h"')

    #run the 'main' function
    main()
```

```
#program options from boost for LPS configuration
gROOT.ProcessLine("boost::program_options::options_description opt(\"opt\");")

#LPS instance, solution dimensionality is 1 (single value)
solver = rt.LookupProblemSolver(1, "Gauss2D", rt.opt)

#independent quantities, x and y of the Gaussian
solver.MakeQuantity("x")
solver.MakeQuantity("y")

#load 'conf.ini' with optional configuration for the layers
gROOT.ProcessLine("boost::program_options::variables_map opt_map;")
gROOT.ProcessLine("boost::program_options::store(parse_config_file(\"conf.ini\", opt), opt_map);")

#optional non-uniform segmentation in quantities
#solver.SetUseMedCellFinder()

#initialize the LPS for training
solver.Initialize(rt.opt_map)
```

```
#linear function to generate values for x and y,  
#non-uniform from 0.3 to 1 over x from -3 to 3  
linear = TF1("linear", "(1.3/2)+(0.7/6)*x", -3, 3)  
  
#number of training values  
nxy = 10000  
  
#training loop  
for i in range(nxy):  
  
    #x and y as independent quantities  
    x = linear.GetRandom()  
    y = linear.GetRandom()  
  
    #Gaussian value as known solution  
    gv = gauss2.Eval(x, y)  
  
    #set the quantities and solution to vector containers for LPS  
    quant = std.vector("double")([x, y])  
    sol = std.vector("double")([gv])  
  
    #add training input to LPS  
    solver.AddInput(quant, sol);
```

```
#finalize the LPS for export
solver.Finalize()

#export to file
out = TFile("lps_gaussian_py.root", "recreate")

solver.Export()

out.Close()

#draw the Gaussian which was used for the training
can = TCanvas("can", "can", 768, 768)

gauss2.Draw("colz")

can.SaveAs("gauss_py.pdf")
```

Solving by LPS for 2D Gaussian (solve.py)

```
#LPS instance without program_options for obtaining solutions
solver = rt.LookupProblemSolver(1, "Gauss2D") #dimensionality and name match training
#import trained LPS
inp = TFile("lps_gaussian_py.root", "read")
solver.Import(inp)

#set the TGraph and nxy here

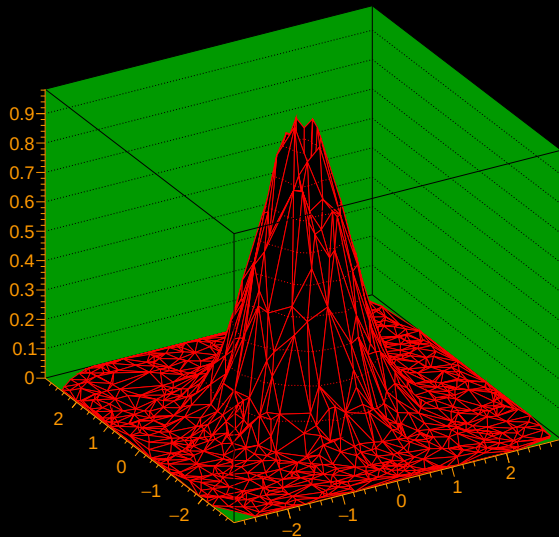
#point loop
for i in range(nxy):

    #random x and y uniformly from -3 to 3
    x = -3 + 6*gRandom.Rndm()
    y = -3 + 6*gRandom.Rndm()
    #put x and y (independent quantities) to vector container
    quant = std.vector("double")([x, y])

    #prepare vector container for solution
    sol = std.vector("double")()
    sol.resize(1) # set length as 1

    #obtain the solution for x and y (passed to 'sol')
    stat = solver.Solve(quant, sol)
    #check status if solution was found
    if stat == False:
        continue
    #set the solution to the graph and then draw it
```

Outcome of training with
10 000 values and solving for
1000 points



- Kinematics for tracks from physics Geant simulation (photoproduction process) will be reconstructed with use of trained LPS
- The training is done with another Geant simulation of electrons with uniform distribution in kinematics
- Template macros are in `TaggerTracks` directory, `track_training.py` and `track_reco.py`
- Data are in: `/data2/vyuka/szd/LookupProblemSolver/data/`

```

#open the TTree input with tracks here

#program options from boost for LPS configuration
gROOT.ProcessLine("boost::program_options::options_description opt(\"opt\");")

#LPS instance, solution dimensionality is 3 (energy, theta, phi)
solver = rt.LookupProblemSolver(3, "Tagger_1", rt.opt)
#independent quantities as track geometry parameters
solver.MakeQuantity("x") # x position, mm
solver.MakeQuantity("y") # y position, mm
solver.MakeQuantity("theta_x") # theta_x angle, rad
solver.MakeQuantity("theta_y") # theta_y angle, rad

#in event loop selecting events with one track:
#get the track (it is known there is one)
trk = tracks.GetUnit(0)

#set independent quantities for LPS from track geometry
quant = std.vector("double")([trk.x, trk.y, trk.theta_x, trk.theta_y])

#set the known solution from true kinematics, length is 3
sol = std.vector("double")([true_en[0], true_theta[0], true_phi[0]])

#add training input to LPS
solver.AddInput(quant, sol)

#finalize and export after the event loop

```

LPS reconstruction for tagger tracks, track_reco.py

```
#LPS instance for obtaining solutions
solver = rt.LookupProblemSolver(3, "Tagger_1")

#import trained LPS
inp_lps = TFile("lps_tracks.root", "read")
solver.Import(inp_lps)

#inside the event loop:

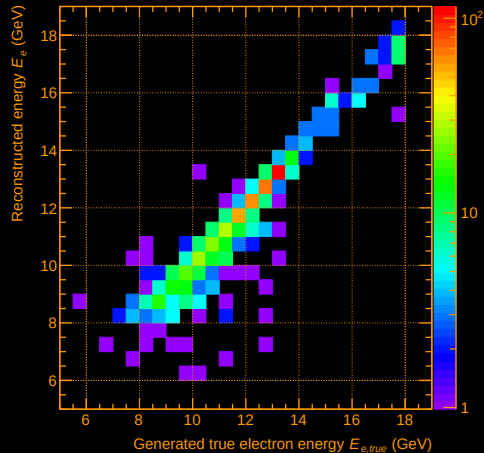
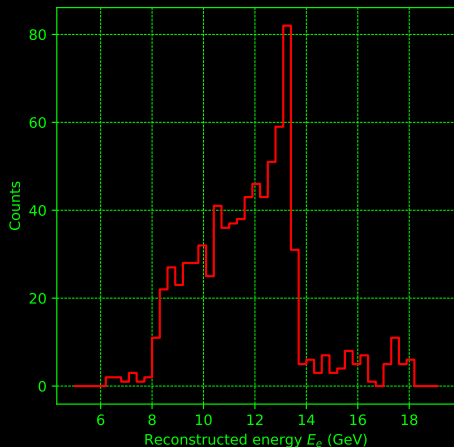
    #tracks loop
    for trk in tracks.GetReadData():

        #independent quantities for LPS from track geometry
        quant = std.vector("double")([trk.x, trk.y, trk.theta_x, trk.theta_y])

        #prepare the vector container for the solutions
        sol = std.vector("double")()
        sol.resize(3) # length is 3

        #obtain the solution and check its status (passed by ref to sol)
        stat = solver.Solve(quant, sol)
        if stat == False: continue

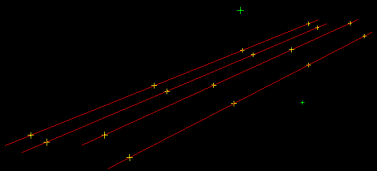
        #put the kinematics solution to output tree and fill it
```



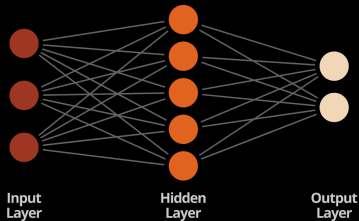
Comparison of LPS reconstructed to true generated electron energy

- Set of 4 measured quantities (track position and angles in detector frame) is connected to 3-dimensional solutions (electron energy, polar and azimuthal angles)
- LPS is trained by providing quantities for known solutions, then set of links from quantities to solutions is formed

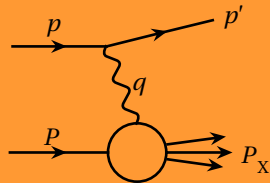
Tagger tracks (4 measured quantities for each track)



Links to solutions in LPS



DIS electron kinematics (3-dimensional solutions)



LPS implementation:

github.com/adamjaro/lmon2/blob/master/reconstruction/src/LookupProblemSolver.cxx

github.com/adamjaro/lmon2/blob/master/reconstruction/include/LookupProblemSolver.h

- Objects of `Link` from quantities to solutions are held in `std::map<ULong64_t, Link>`
- Every particular set of quantities is assigned a unique `ULong64_t` integer number (64 bit unsigned integer, longest in ROOT)
- Values for each quantity are binned, for m quantities number of bins are $n_0, n_1, n_2, \dots, n_{m-1}$
- Bin indices for particular quantities values are $i_0, i_1, i_2, \dots, i_{m-1}$
- Each i_r takes values from 1 to n_r , convention of `TH1D` (r runs from 0 to $m - 1$)
- Unique `ULong64_t` index i is obtained from bin indices as:

$$i = i_0 + i_1 * n_0 + i_2 * n_0 * n_1 + i_3 * n_0 * n_1 * n_2 + \dots + i_{m-1} * n_0 * n_1 * n_2 \dots * n_{m-1}$$

- More layers can be formed for various binnings n_r and schemes for bin indices i_r
- The schemes are formed with fixed bin size (default with `TH1D`) or variable bin sizes using `TFoam`, median interval splitting or splitting for same number of entries in each bin
- Solutions with uncertainties are stored in each `Link` as `std::vector<double>`, k -dimensional solution needs `std::vector` of size k (one for solution values and another for uncertainties)

- During training, objects of `Link` are created in the `std::map` at given indices i and filled with input solutions (function `LookupProblemSolver::AddInput`)
- Input solutions are stored in each `Link` as `std::vector<std::vector<double>>`, inner `std::vector` has size k for k -dimensional solution, outer `std::vector` has its size given by number of input solutions provided at the respective index i
- When finishing the training, most probable solution for each `Link` is calculated as arithmetic mean of all inputs provided at given index i with associated uncertainty
- Trained `std::map` is exported as `TTree` and then can be imported to run the reconstruction
- Reconstruction with trained `std::map<ULong64_t, Link>` means providing set of quantities, calculating `ULong64_t` index i and retrieving the solution from the `std::map`
- The reconstruction is implemented in function `LookupProblemSolver::Solve`, returns `true` when the solution was found for the provided quantities (and pass-by-reference for solution values)
- Selection criteria for solution uncertainties or minimal number of inputs can be provided to get the solution from the best matching layer

- Full codes used today are in <https://github.com/adamjaro/lmon2/tree/master/examples/LookupProblemSolver>
- Books listed below are in </data2/vyuka/szd/LookupProblemSolver/References/>
- C++ in general
 - Eckel, Thinking in C++, volume 1 and 2, 2000
 - Meyers, Effective Modern C++, 2015
- Python and PyROOT
 - Lott, Phillips, Python Object-Oriented Programming, 2021
 - [JA-Python_20260214.pdf](#)
- Machine learning
 - Calafiura et al. (eds.), Artificial Intelligence for High Energy Physics, 2022
 - Labonne, Graph Neural Networks Using Python, 2023
- History of ML
 - Alt, Rubinoff (eds.), Advances in Computers, 1964
 - Alt, Rubinoff (eds.), Advances in Computers, 1965